Отчёт о защите капстоуна

Дисциплина «Системное администрирование», 1 семестр

Студент: Светличный Никита Юрьевич

Репозиторий: https://github.com/SvetlichnyiNikita/my-sysadmin-scripts

Дорожка: «База» (в ДЗ1 сделал ещё команды уровня «Продвинутый», в ДЗ2 - бонус с расширением LVM-тома)

Сервер: облако MakeCloud, Ubuntu 22.04, 1 vCPU, 1 ГБ RAM, 10 ГБ диск, хост sysadmin-host

Когда снят вывод: 17.10.2026

Что в итоге сделано

Мой скрипт мониторинга (вариант Б из ДЗ1) каждые 5 секунд записывает в файл monitor.log вывод `free -h`, `df -h` и `uptime` со временем. Скрипт я упаковал в Docker. В контейнере вместе со скриптом запущен простой веб-сервер на python, он отдаёт этот лог по порту 8080. Перед контейнером стоит nginx: он работает по https на 443 порту, а с 80 порта перекидывает на https. Контейнер запускается через systemd-службу my-app. Ещё на сервере собраны RAID 1 и LVM на loop-устройствах (это файлы, которые система видит как диски).

```
запрос -> nginx (443, https) -> 127.0.0.1:8080 -> контейнер my-app (python + script.sh) -> monitor.log
                                                  ^ запускается службой my-app через systemd
```

Все команды ниже я запускал на сервере одним скриптом и сохранял вывод в файл, сюда он вставлен как есть. Убрал только лишние строки, которые curl печатает при записи в файл, и в пункте 2 оставил один блок лога из трёх, там это отмечено.

Пункты чек-листа

1. Git-репозиторий

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ git log --oneline
34c52b4 deploy: конфиги nginx и systemd, обновил README
1c292ed docker: контейнер отдаёт monitor.log по http на 8080
03ddb02 script: путь к логу можно задать через LOG_FILE
1f947de docker: добавил docker-compose.yml
1b0b98f docker: добавил Dockerfile для скрипта
5fa05e9 docs: добавлен README и пример вывода
2e9d46b fix: проверка прав на лог и корректная остановка
b7642f7 draft: первая версия скрипта мониторинга

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ git remote -v
origin  git@github.com:SvetlichnyiNikita/my-sysadmin-scripts.git (fetch)
origin  git@github.com:SvetlichnyiNikita/my-sysadmin-scripts.git (push)
```

Репозиторий открытый, на момент проверки в нём 8 коммитов. Потом я добавил ещё один, когда исправлял порт 8080 (про это в конце отчёта). По коммитам видно, как я делал работу: сначала черновик скрипта, потом исправления и README, потом Docker и настройка сервера. В README написано, что делает скрипт и как его запустить.

2. Скрипт

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ rm -f /tmp/test.log; LOG_FILE=/tmp/test.log timeout 12 ./scri
    pt.sh
Мониторинг запущен, пишу в /tmp/test.log каждые 5 сек. Остановка: Ctrl+C
Terminated

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ cat /tmp/test.log
--- 2026-10-17 11:10:15 ---
               total        used        free      shared  buff/cache   available
Mem:           952Mi       262Mi        79Mi       1.0Mi       609Mi       525Mi
Swap:             0B          0B          0B
Filesystem                  Size  Used Avail Use% Mounted on
```

```
tmpfs                          96M  1.3M   95M   2% /run
/dev/sda1                     9.6G  4.6G  5.0G  48% /
tmpfs                         477M     0  477M   0% /dev/shm
tmpfs                         5.0M     0  5.0M   0% /run/lock
/dev/sda15                    105M  6.1M   99M   6% /boot/efi
/dev/md0                      463M   24K  427M   1% /mnt/raid
/dev/mapper/vg_data-lv_logs  266M   24K  250M   1% /mnt/logs
tmpfs                          96M  4.0K   96M   1% /run/user/1000
 11:10:15 up 10 days, 18:25,  1 user,  load average: 0.44, 0.15, 0.04

[... дальше ещё два таких же блока: 11:10:20 и 11:10:25 ...]

--- 2026-10-17 11:10:27 мониторинг остановлен ---
```

Запустил скрипт на 12 секунд, а лог записал во временный файл, чтобы не смешивать с основным. Скрипт успел сделать три записи: в 11:10:15, 11:10:20 и 11:10:25, то есть раз в 5 секунд, как и должно быть. Когда время вышло, скрипт сам дописал строку «мониторинг остановлен». Слово Terminated - это не ошибка, так timeout сообщает, что остановил скрипт.

3. Docker

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ docker build -t my-script .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  116.7kB

Step 1/8 : FROM ubuntu:22.04
 ---> 5ec03bb3441e
Step 2/8 : RUN apt-get update && apt-get install -y --no-install-recommends python3     && rm -rf /var/lib/apt/l
    ists/*
 ---> Using cache
 ---> 9da27fa82e7b
Step 3/8 : WORKDIR /var/www
 ---> Using cache
 ---> 83a9ec190e4d
Step 4/8 : COPY script.sh /usr/local/bin/script.sh
 ---> Using cache
 ---> 01757acd4af3
Step 5/8 : RUN chmod +x /usr/local/bin/script.sh
 ---> Using cache
 ---> fd7b35fb4003
Step 6/8 : ENV LOG_FILE=/var/www/monitor.log
 ---> Using cache
 ---> a61b6cb2173d
Step 7/8 : EXPOSE 8080
 ---> Using cache
 ---> 45ea5ae80a98
Step 8/8 : CMD ["/bin/bash", "-c", "/usr/local/bin/script.sh & exec python3 -m http.server 8080"]
 ---> Using cache
 ---> 9d621af14930
Successfully built 9d621af14930
Successfully tagged my-script:latest

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ docker run -d --name capstone-test my-script
5a545bde914eaa544bae8fb11fdf006ca671dc665006e7c36bd5c27744dc1d28

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED         STATUS         PORTS
                      NAMES
5a545bde914e   my-script   "/bin/bash -c '/usr/…"   8 seconds ago   Up 7 seconds   8080/tcp
                      capstone-test
74d2c5d34821   my-script   "/bin/bash -c '/usr/…"   10 days ago     Up 10 days     0.0.0.0:8080->8080/tcp, [::]:
    8080->8080/tcp   my-app

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ docker exec capstone-test tail -n 14 /var/www/monitor.log
               total        used        free      shared  buff/cache   available
Mem:           952Mi       261Mi        73Mi       1.0Mi       618Mi       527Mi
Swap:             0B          0B          0B
Filesystem      Size  Used Avail Use% Mounted on
overlay         9.6G  4.6G  5.0G  48% /
tmpfs            64M     0   64M   0% /dev
shm              64M     0   64M   0% /dev/shm
/dev/sda1       9.6G  4.6G  5.0G  48% /etc/hosts
tmpfs           477M     0  477M   0% /proc/acpi
tmpfs           477M     0  477M   0% /proc/scsi
tmpfs           477M     0  477M   0% /sys/devices/virtual/powercap
tmpfs           477M     0  477M   0% /sys/firmware
```

```
 08:10:32 up 10 days, 18:26,  0 users,  load average: 0.31, 0.14, 0.04

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ docker rm -f capstone-test
capstone-test
```

Образ собирается без ошибок (Using cache значит, что Dockerfile не менялся и docker взял готовые шаги). Чтобы не трогать рабочий контейнер, запустил для проверки отдельный, capstone-test. Он в статусе Up, и внутри уже пишется лог. Второй контейнер в списке, my-app, - это рабочий. Время в логе контейнера на 3 часа меньше, потому что в контейнере время по UTC, а не московское. Надпись DEPRECATED - просто предупреждение, на сборку оно не влияет.

4. RAID и LVM

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ cat /proc/mdstat
Personalities : [linear] [multipath] [raid0] [raid1] [raid6] [raid5] [raid4] [raid10]
md0 : active raid1 loop4[1] loop3[0]
      523264 blocks super 1.2 [2/2] [UU]

unused devices: <none>

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ sudo vgs
  VG      #PV #LV #SN Attr   VSize   VFree
  vg_data   1   1   0 wz--n- 508.00m 208.00m

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ sudo lvs
  LV      VG      Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_logs vg_data -wi-ao---- 300.00m

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ df -h /mnt/raid /mnt/logs
Filesystem                  Size  Used Avail Use% Mounted on
/dev/md0                    463M   24K  427M   1% /mnt/raid
/dev/mapper/vg_data-lv_logs  266M   24K  250M   1% /mnt/logs
```

md0 - это RAID 1 из двух дисков по 512 МБ (loop3 и loop4). [UU] значит, что оба диска работают. На третьем диске сделал LVM: группа vg_data и в ней том lv_logs. Сначала том был 200 МБ, потом я увеличил его до 300 МБ командой `lvextend -r -L +100M`, не отключая его. Оба тома подключены: /mnt/raid и /mnt/logs.

5. Nginx

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ ls /etc/nginx/sites-enabled/
my-app

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Мой конфиг (в репозитории лежит в deploy/nginx-my-app.conf):

```
server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    ssl_certificate     /etc/ssl/certs/my-app.crt;
    ssl_certificate_key /etc/ssl/private/my-app.key;

    location / {
        proxy_pass http://127.0.0.1:8080;
    }
}
```

`nginx -t` проходит без ошибок. Включён только мой конфиг, стандартный default я удалил, потому что с ним nginx ругался на одинаковый server_name. Первый блок отвечает за 80 порт и перекидывает на https, второй принимает https и передаёт запросы в контейнер на 8080.

6. HTTPS

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ openssl x509 -in /etc/ssl/certs/my-app.crt -noout -subject -i
    ssuer -dates
subject=CN = my-app.local
issuer=CN = my-app.local
notBefore=Oct  6 17:41:41 2026 GMT
notAfter=Oct  6 17:41:41 2027 GMT

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ curl -kI https://10.0.1.4
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 17 Oct 2026 08:10:35 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 344
Connection: keep-alive

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ curl -I http://10.0.1.4
HTTP/1.1 301 Moved Permanently
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 17 Oct 2026 08:10:35 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://10.0.1.4/
```

И то же самое со своего компьютера по внешнему адресу сервера:

```
PS C:\Users\User> curl.exe -m 5 -kI https://45.67.32.134
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 17 Oct 2026 08:14:06 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 344
Connection: keep-alive
```

Сертификат я сделал сам через openssl, поэтому в subject и issuer одно и то же имя. Действует год, до октября 2027. По https сервер отвечает 200, по http - 301 и перекидывает на https. Флаг `-k` нужен, потому что сертификат самодельный и curl ему без этого флага не доверяет. 10.0.1.4 - это адрес сервера внутри облака, 45.67.32.134 - внешний.

7. systemd

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ systemctl is-enabled my-app
enabled

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ systemctl status my-app --no-pager
● my-app.service - my-app service
     Loaded: loaded (/etc/systemd/system/my-app.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2026-10-06 20:42:46 MSK; 1 week 3 days ago
   Main PID: 8192 (docker)
      Tasks: 8 (limit: 1094)
     Memory: 7.6M
        CPU: 4.050s
     CGroup: /system.slice/my-app.service
             └─8192 docker start -a my-app

Oct 17 11:06:10 sysadmin-host docker[8192]: 199.165.159.53 - - [17/Oct/2026 08:06:10] "GET /favicon.ico HTTP/1.1
    " 404 -
Oct 17 11:07:02 sysadmin-host docker[8192]: 199.165.159.52 - - [17/Oct/2026 08:07:02] code 404, message File not
     found
Oct 17 11:07:02 sysadmin-host docker[8192]: 199.165.159.52 - - [17/Oct/2026 08:07:02] "GET http://api.ipify.org/
    ?format=json HTTP/1.1" 404 -
Oct 17 11:07:19 sysadmin-host docker[8192]: 199.165.159.53 - - [17/Oct/2026 08:07:19] code 501, message Unsuppor
    ted method ('CONNECT')
Oct 17 11:07:19 sysadmin-host docker[8192]: 199.165.159.53 - - [17/Oct/2026 08:07:19] "CONNECT www.shadowserver.
    org:443 HTTP/1.1" 501 -
Oct 17 11:07:41 sysadmin-host docker[8192]: 199.165.159.49 - - [17/Oct/2026 08:07:41] code 404, message File not
     found
Oct 17 11:07:41 sysadmin-host docker[8192]: 199.165.159.49 - - [17/Oct/2026 08:07:41] "GET /geoserver/web/ HTTP/
    1.1" 404 -
Oct 17 11:09:54 sysadmin-host docker[8192]: 172.17.0.1 - - [17/Oct/2026 08:09:54] "HEAD / HTTP/1.0" 200 -
Oct 17 11:09:54 sysadmin-host docker[8192]: 172.17.0.1 - - [17/Oct/2026 08:09:54] "HEAD /monitor.log HTTP/1.0" 2
    00 -
Oct 17 11:10:35 sysadmin-host docker[8192]: 172.17.0.1 - - [17/Oct/2026 08:10:35] "HEAD / HTTP/1.0" 200 -
```

Файл службы (в репозитории deploy/my-app.service):

```
[Unit]
Description=my-app service
After=docker.service
Requires=docker.service

[Service]
ExecStart=docker start -a my-app
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Служба включена (enabled), значит запустится сама после перезагрузки сервера. Работает без остановок с 6 октября. Строчки After и Requires нужны, чтобы служба стартовала только после Docker. Внизу вывода видны запросы с чужих IP-адресов - из-за них я нашёл проблему, про неё в конце отчёта.

8. Логи

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ curl -skI https://localhost/monitor.log
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 17 Oct 2026 08:10:35 GMT
Content-Type: application/octet-stream
Content-Length: 137355275
Connection: keep-alive
Last-Modified: Sat, 17 Oct 2026 08:10:31 GMT

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ sudo journalctl -u my-app -n 5 --no-pager
Oct 17 11:07:41 sysadmin-host docker[8192]: 199.165.159.49 - - [17/Oct/2026 08:07:41] "GET /geoserver/web/ HTTP/
    1.1" 404 -
Oct 17 11:09:54 sysadmin-host docker[8192]: 172.17.0.1 - - [17/Oct/2026 08:09:54] "HEAD / HTTP/1.0" 200 -
Oct 17 11:09:54 sysadmin-host docker[8192]: 172.17.0.1 - - [17/Oct/2026 08:09:54] "HEAD /monitor.log HTTP/1.0" 2
    00 -
Oct 17 11:10:35 sysadmin-host docker[8192]: 172.17.0.1 - - [17/Oct/2026 08:10:35] "HEAD / HTTP/1.0" 200 -
Oct 17 11:10:35 sysadmin-host docker[8192]: 172.17.0.1 - - [17/Oct/2026 08:10:35] "HEAD /monitor.log HTTP/1.0" 2
    00 -

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ sudo tail -n 3 /var/log/nginx/access.log
10.0.1.4 - - [17/Oct/2026:11:10:35 +0300] "HEAD / HTTP/1.1" 200 0 "-" "curl/7.81.0"
10.0.1.4 - - [17/Oct/2026:11:10:35 +0300] "HEAD / HTTP/1.1" 301 0 "-" "curl/7.81.0"
127.0.0.1 - - [17/Oct/2026:11:10:35 +0300] "HEAD /monitor.log HTTP/1.1" 200 0 "-" "curl/7.81.0"

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ sudo tail -n 3 /var/log/nginx/error.log
2026/10/17 03:15:04 [crit] 7966#7966: *7857 SSL_do_handshake() failed (SSL: error:0A00006C:SSL routines::bad key
     share) while SSL handshaking, client: 199.165.159.53, server: 0.0.0.0:443
2026/10/17 06:52:29 [crit] 7966#7966: *8067 SSL_do_handshake() failed (SSL: error:0A00006C:SSL routines::bad key
     share) while SSL handshaking, client: 135.136.68.25, server: 0.0.0.0:443
2026/10/17 09:49:44 [crit] 7966#7966: *8170 SSL_do_handshake() failed (SSL: error:0A00006C:SSL routines::bad key
     share) while SSL handshaking, client: 20.121.140.118, server: 0.0.0.0:443
```

Сделал тестовый запрос к monitor.log через https. Он сразу появился в двух логах: в журнале службы my-app (последняя строка) и в access.log nginx (тоже последняя строка). В журнале службы адрес 172.17.0.1 - это значит, что запрос пришёл от nginx. В error.log ошибки при подключении к 443 порту с чужих адресов. Похоже, это боты из интернета, к моей настройке это не относится. Ещё по Content-Length видно, что monitor.log за 10 дней вырос до 137 МБ.

Почему сделал именно так

Почему вариант Б

В первой части Д31 я уже смотрел free, df и uptime руками, и было логично сделать скрипт, который делает то же самое сам и сохраняет историю. Ещё ему не нужны права root и доступ к системным логам, поэтому в контейнере он заработал без дополнительной настройки. И проверить его просто: открыл лог и видно, пишется он или нет.

RAID 1 и LVM

Сделал оба, потому что они нужны для разного.

RAID 1 нужен для надёжности: одни и те же данные пишутся на два диска сразу, и если один сломается, всё останется на втором. Но место из-за этого теряется: у меня два диска по 512 МБ, а места получилось около 460 МБ, как на одном.

LVM нужен, чтобы можно было менять размер тома. Данные он не копирует. Я проверил это на деле: сделал том на 200 МБ, потом увеличил до 300 МБ одной командой, ничего не отключая.

Почему самоподписанный сертификат

Бесплатный сертификат от Let's Encrypt выдают только на домен, а у меня домена нет, только IP-адрес. Поэтому сделал сертификат сам через openssl. Шифрование при этом работает так же, но браузер будет предупреждать, что сертификат не проверенный. Если бы был домен, взял бы Let's Encrypt.

Какие были другие варианты

Локальная виртуалка вместо облака. Можно было поставить Multipass у себя, но я выбрал облачный сервер: он работает всегда и на нём можно делать все ДЗ подряд. Минус - он платный, около 5 рублей в день.

Без nginx. Можно было открыть порт 8080 наружу напрямую, но python-сервер не умеет https. Через nginx есть https, перенаправление с http и нормальный лог запросов.

Без systemd. У docker есть свой автозапуск контейнеров (`--restart`), но в задании нужна systemd-служба. Так служба видна в systemctl status, а её логи - в journalctl.

Путь к логу. Можно было просто прописать в скрипте путь /var/www, но тогда скрипт перестал бы нормально работать вне контейнера. Поэтому сделал переменную LOG_FILE: в контейнере путь задаётся в Dockerfile, а без неё лог пишется рядом со скриптом, как раньше.

Бонусы ДЗ2 (multi-stage build и загрузка образа в GHCR). Делать не стал. Multi-stage нужен, когда программу надо компилировать, а у меня скрипт на bash. Образ собирается за пару секунд прямо на сервере, поэтому выкладывать его куда-то не было смысла.

Что не получилось с первого раза

1. Лог в контейнере. Скрипт пишет monitor.log в ту же папку, где лежит сам. В контейнере это /usr/local/bin, а python-сервер отдаёт файлы из /var/www, так что открыть лог через браузер не получилось бы. Поэтому сделал переменную LOG_FILE и в Dockerfile указал путь /var/www/monitor.log. Для этого есть отдельный коммит с доработкой скрипта.

2. 200 вместо 301. Когда настроил https, проверил `curl -I http://localhost` и получил 200 и стандартную страницу nginx, хотя должен был быть редирект. Проверил, что включён только мой конфиг, повторил запрос, и пришёл 301. Скорее всего, я проверил слишком быстро, и nginx ещё не успел перечитать новый конфиг.

3. Порт 8080 был открыт наружу. Это я заметил, когда готовил отчёт. В `systemctl status my-app` (пункт 7) были запросы с незнакомых адресов, например GET /geoserver/web/. Это боты из интернета, и они шли прямо в контейнер, мимо nginx: запросы через nginx приходят с адреса 172.17.0.1, а у этих были свои адреса. Оказалось, что `docker run -p 8080:8080` открывает порт на всех адресах сервера, то есть и для интернета тоже. А я думал, что снаружи к серверу можно подключиться только по SSH.

Исправил так: пересоздал контейнер с `-p 127.0.0.1:8080:8080`. Теперь порт 8080 доступен только с самого сервера, а снаружи можно зайти только через nginx по https. Поправил команду в README и сделал отдельный коммит. Проверка после исправления:

```
n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED          STATUS          PORTS
```

```
    NAMES
16b570ab3abf   my-script   "/bin/bash -c '/usr/…"   25 seconds ago   Up 14 seconds   127.0.0.1:8080->8080/tcp
    my-app

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ sudo ss -tlnp | grep -E ':(80|443|8080) '
LISTEN 0      511          0.0.0.0:80         0.0.0.0:*    users:(("nginx",pid=7966,fd=6),("nginx",pid=7838,fd=6
    ))
LISTEN 0      4096       127.0.0.1:8080       0.0.0.0:*    users:(("docker-proxy",pid=1167991,fd=7))
LISTEN 0      511          0.0.0.0:443        0.0.0.0:*    users:(("nginx",pid=7966,fd=11),("nginx",pid=7838,fd=
    11))

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ systemctl status my-app --no-pager | head -5
● my-app.service - my-app service
     Loaded: loaded (/etc/systemd/system/my-app.service; enabled; vendor preset: enabled)
     Active: active (running) since Sat 2026-10-17 11:13:39 MSK; 14s ago
   Main PID: 1167928 (docker)
      Tasks: 7 (limit: 1094)

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ curl -skI https://localhost
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 17 Oct 2026 08:13:53 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 344
Connection: keep-alive

n_svetlichnyi@sysadmin-host:~/my-sysadmin-scripts$ git log --oneline | head -3
f97d91a fix: порт 8080 только на localhost, снаружи заходим через nginx
34c52b4 deploy: конфиги nginx и systemd, обновил README
1c292ed docker: контейнер отдаёт monitor.log по http на 8080
```

И с моего компьютера: по https сайт открывается, а на порт 8080 подключиться больше нельзя.

```
PS C:\Users\User> curl.exe -m 5 -kI https://45.67.32.134
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 17 Oct 2026 08:14:06 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 344
Connection: keep-alive

PS C:\Users\User> curl.exe -m 5 -I http://45.67.32.134:8080
curl: (28) Operation timed out after 5011 milliseconds with 0 bytes received
```

Что ещё можно доделать

Лог monitor.log растёт без остановки: за 10 дней уже 137 МБ. Надо сделать так, чтобы старые записи удалялись, иначе он когда-нибудь займёт весь диск. И если появится домен, поставить нормальный сертификат вместо самодельного.